

Sri Lanka Institute of Information Technology



BSc (Hons) in Information Technology

Specializing in Data Science

IT3021 – Data Warehousing and Business Intelligence

Year 3

Assignment 1

Semester 1, 2026

Magedaragama M.M - IT23837980

Table Of Contents

1. Introduction

2. Data Set Selection

- 2.1 Dataset Description
- 2.2 OLTP Characteristics
- 2.3 Entities and Relationships
- 2.4 ER Diagram

3. Preparation of Data Sources

- 3.1 Overview of Source Systems
- 3.2 SQL Server Source (Accident & Vehicle Data)
- 3.3 CSV Source (Location Data)
- 3.4 Excel Source (Vehicle Category Data)
- 3.5 Data Source Integration Strategy
- 3.6 Data Source Summary

4. Solution Architecture

- 4.1 Architecture Overview
- 4.2 Architectural Diagram
- 4.3 Component Description

5. Data Warehouse Design & Development

- 5.1 Dimensional Model Overview
- 5.2 Star Schema Design
- 5.3 Fact Table – FactVehicleAccident
- 5.4 Dimension Tables
 - 5.4.1 DimDate
 - 5.4.2 DimLocation
 - 5.4.3 DimRoadEnvironment
 - 5.4.4 DimVehicle (SCD Type 2)
 - 5.4.5 DimDriver
 - 5.4.6 DimSeverity
- 5.5 Relational Diagram
- 5.6 Design Assumptions

6. ETL Development

- 6.1 ETL Overview
- 6.2 Connection Managers

- 6.3 Package 1: 01_Load_Dimensions.dtsx
 - 6.3.1 Dimension Loading Strategy
 - 6.3.2 Loading DimDate
 - 6.3.3 Loading DimLocation
 - 6.3.4 Loading DimRoadEnvironment
 - 6.3.5 Loading DimVehicle (SCD Type 2)
 - 6.3.6 Loading DimDriver
 - 6.3.7 Loading DimSeverity
- 6.4 Package 2: 02_Load_FactVehicleAccident.dtsx
 - 6.4.1 Source Data Extraction
 - 6.4.2 Data Transformations
 - Derived Columns
 - Lookups
 - 6.4.3 Fact Table Loading Process
- 6.5 Execution Results

7. Accumulating Fact Table Implementation

- 7.1 Concept of Accumulating Fact Table
- 7.2 Fact Table Extension (New Columns)
- 7.3 Initial Load Enhancements (txn_id & create time)
- 7.4 Update Dataset Preparation (CSV File)
- 7.5 Staging Table Design
- 7.6 Stored Procedure Implementation
- 7.7 Package 3: 03_Update_AccumulatingFact.dtsx
 - 7.7.1 Control Flow Design
 - 7.7.2 Data Flow – Load CSV to Staging
 - 7.7.3 Data Flow – Update Fact Table
- 7.8 Validation and Results
 - 7.8.1 Updated Fact Table Records
 - 7.8.2 Process Time Calculation Verification

8. Conclusion

Introduction

In today's data-driven world, large volumes of data are continuously generated from real-world operational systems. In domains such as transportation and public safety, this data plays a crucial role in understanding patterns, improving infrastructure, and supporting strategic decision-making. However, raw data collected from multiple sources is often fragmented, heterogeneous, and not optimized for analytical purposes. Data Warehousing and Business Intelligence (DWBI) solutions address this challenge by integrating, transforming, and organizing data into a structured format suitable for analysis and reporting.

This assignment focuses on designing and implementing a complete data warehousing solution using the **UK Road Safety dataset**, which contains detailed records of road traffic accidents and vehicle information across the United Kingdom. The dataset represents a real-world OLTP system, where accident events and related vehicle details are recorded in a normalized structure. To simulate a realistic enterprise environment, data is sourced from multiple formats, including a SQL Server database, CSV files, and Excel files.

The implementation of this project is carried out through several stages, including dataset selection, preparation of data sources, solution architecture design, dimensional modeling using a star schema, and ETL development using SQL Server Integration Services (SSIS). The data warehouse is designed around a central fact table (**FactVehicleAccident**) supported by multiple dimension tables such as **DimDate**, **DimLocation**, **DimRoadEnvironment**, **DimVehicle**, **DimDriver**, and **DimSeverity**, enabling efficient analytical processing.

Advanced data warehousing concepts are also incorporated into the solution. These include the implementation of Slowly Changing Dimensions (Type 2) to manage historical changes in vehicle data, as well as an accumulating fact table to track transaction lifecycle events. The ETL process integrates data from heterogeneous sources, performs data cleansing and transformation using lookups and derived columns, and ensures consistency and integrity before loading into the data warehouse.

The final outcome of this assignment is a fully functional data warehouse that supports analytical queries related to accident trends, environmental conditions, and vehicle characteristics. This demonstrates how raw operational data can be transformed into meaningful insights, enabling better decision-making in the context of road safety and transportation management.

2. Data Set Selection

2.1 Dataset Description

The dataset selected for this assignment is the **UK Road Safety Accidents and Vehicles dataset**, obtained from Kaggle. This dataset captures detailed records of road traffic accidents across the United Kingdom and includes both accident-level and vehicle-level information.

The dataset represents a realistic operational environment where transportation and safety-related data is continuously recorded. It consists of multiple interrelated tables that store information about accident events, involved vehicles, environmental conditions, and severity levels. The data spans multiple years, ensuring sufficient volume and temporal coverage required for analytical processing.

Key characteristics of the dataset include:

- Large-scale, real-world data reflecting actual road incidents
- Multiple attributes describing accident conditions, location, and vehicle details
- Strong relationships between entities such as accidents and vehicles
- Temporal data enabling time-based analysis (year, date, time)
- Heterogeneous attributes suitable for dimensional modeling

This dataset is particularly suitable for a data warehousing solution as it allows analysis of accident patterns, environmental impacts, and vehicle-related risk factors.

2.2 OLTP Characteristics

The selected dataset exhibits typical characteristics of an **OLTP (Online Transaction Processing) system**, where data is stored in a normalized structure optimized for transactional operations rather than analytical queries.

The OLTP properties of the dataset are as follows:

- **Highly normalized structure**
 - Data is separated into multiple related entities such as Accident and Vehicle
 - Reduces redundancy and ensures consistency
- **Transactional nature**
 - Each record represents a real-world event (accident occurrence)
 - Vehicle records are linked to specific accidents
- **Use of primary and foreign keys**
 - Accident_Index uniquely identifies each accident
 - Vehicle_Reference identifies vehicles within an accident
 - Relationships are maintained through foreign keys
- **Frequent data updates and inserts**
 - New accident records are continuously generated
 - Vehicle and environmental details are recorded per event
- **Not optimized for analytics**
 - Complex joins are required to retrieve meaningful insights
 - Lack of aggregated or pre-processed data

Due to these characteristics, the dataset is not directly suitable for reporting and analytics, making it an ideal candidate for transformation into a dimensional data warehouse.

2.3 Entities and Relationships

The dataset consists of several core entities that represent different aspects of road traffic accidents. These entities are interconnected to form a structured relational model.

Main Entities

- **Accident**
 - Represents a single road accident event
 - Contains details such as date, time, severity, weather conditions, road conditions, and location
- **Vehicle**
 - Represents each vehicle involved in an accident
 - Includes attributes such as vehicle type, age, engine capacity, and propulsion type
- **Driver (Derived/Logical Entity)**
 - Represents driver-related attributes extracted from vehicle data
 - Includes age band, gender, and driving conditions
- **Location (Derived Entity)**
 - Represents geographical attributes such as district, urban/rural classification, latitude, and longitude
- **Road Environment (Derived Entity)**
 - Represents environmental conditions including weather, road surface, light conditions, and road type
- **Severity (Derived Entity)**
 - Represents accident severity classification (e.g., slight, serious, fatal)

Relationships Between Entities

- **Accident → Vehicle (1:M)**
 - One accident can involve multiple vehicles
 - Each vehicle belongs to exactly one accident
- **Vehicle → Driver (1:1 or 1:M logical mapping)**
 - Each vehicle is associated with a driver profile
- **Accident → Location (M:1)**
 - Multiple accidents can occur in the same location
- **Accident → Road Environment (M:1)**
 - Environmental conditions are shared across multiple accidents
- **Accident → Severity (M:1)**
 - Each accident is classified into a severity category

These relationships demonstrate a well-structured transactional model, which can later be transformed into a **star schema** for analytical processing.

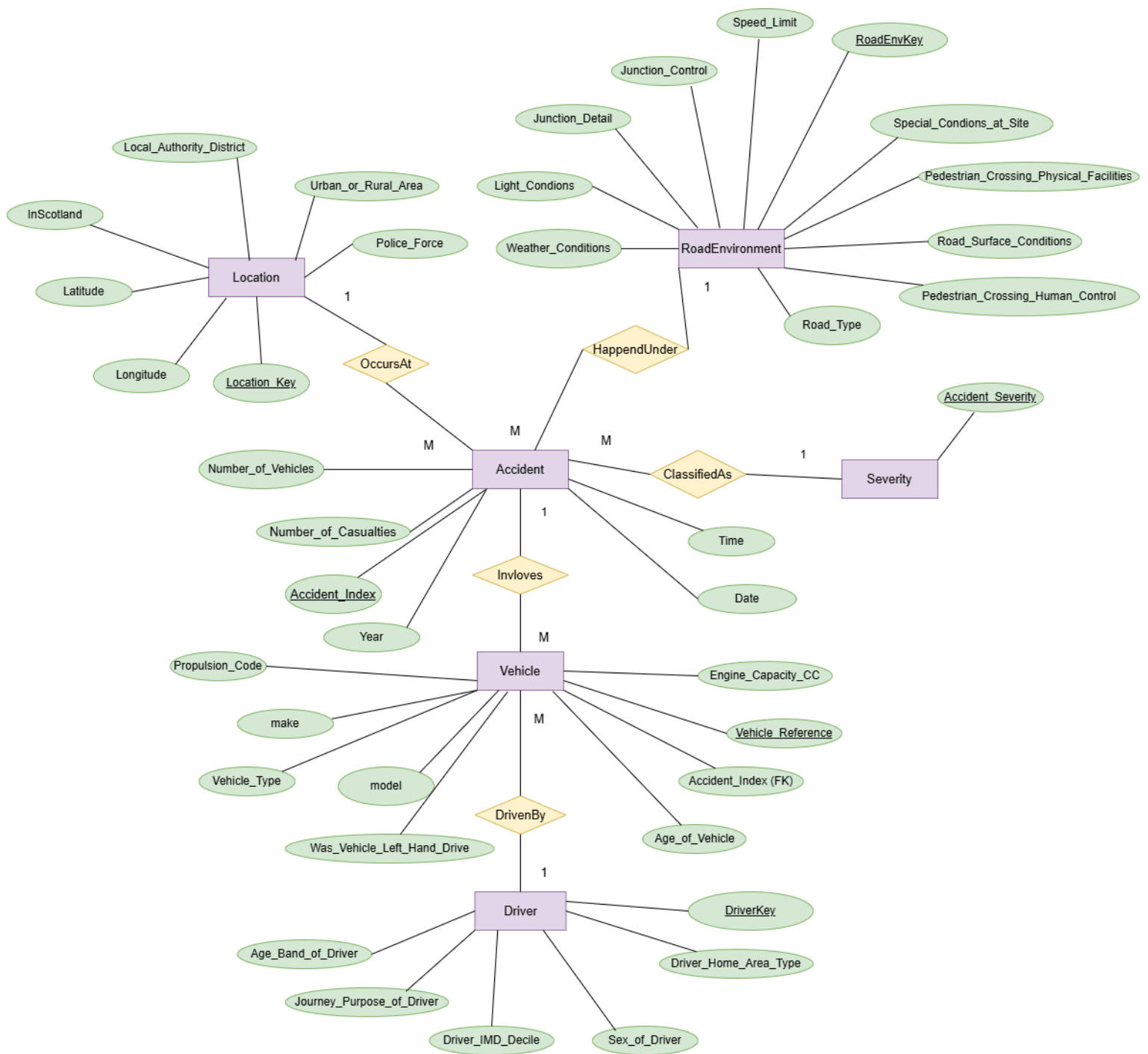
2.4 ER Diagram

The Entity–Relationship (ER) diagram for the Road Safety dataset visually represents the structure of the source system, including entities, attributes, and relationships.

Diagram Description

The ER diagram includes:

- **Primary Entity: Accident**
 - Central entity representing accident events
 - Connected to multiple related entities
- **Vehicle Entity**
 - Linked to Accident via Accident_Index
 - Represents multiple vehicles per accident
- **Derived Logical Entities**
 - Location
 - Road Environment
 - Severity
 - Driver
- **Key Relationships**
 - One-to-Many relationship between Accident and Vehicle
 - Many-to-One relationships between Accident and descriptive entities



Entity- Relationship Diagram of the UK Road Safety dataset

3. Preparation of Data Sources

3.1 Overview of Source Systems

To simulate a realistic enterprise data warehousing environment, the selected Road Safety dataset was prepared and organized into multiple heterogeneous data sources. This approach reflects real-world scenarios where data originates from different systems, formats, and structures, requiring integration through ETL processes.

The dataset was deliberately divided into three distinct source types: a structured relational database, a semi-structured CSV file, and an Excel-based reference file. Each source provides a specific subset of information required for building the data warehouse. This separation ensures that the ETL process demonstrates the ability to extract and integrate data from multiple formats while maintaining consistency and data integrity.

The SQL Server database serves as the primary transactional source, containing accident and vehicle data in a normalized format. In addition, external files were introduced to enrich the dataset and support hierarchical and dimensional analysis. The CSV file provides location-related attributes, while the Excel file contains vehicle categorization details that are not directly available in the core transactional data.

This multi-source design ensures that the solution meets the assignment requirement of handling heterogeneous data sources while enabling complex transformations and integration within the data warehouse.

3.2 SQL Server Source (Accident & Vehicle Data)

The primary data source for this project is a SQL Server database named RoadSafety_SourceDB, which contains the core transactional data. This database represents an OLTP-style system where accident events and vehicle information are stored in a normalized structure.

The two main tables within this database are:

- **Accident_Information**
This table stores accident-level data, where each record represents a single road traffic accident. It includes attributes such as date, time, accident severity, weather conditions, road characteristics, and geographic details. The table uses Accident_Index as the primary key, ensuring that each accident is uniquely identified.
- **Vehicle_Information_Clean**
This table stores vehicle-level data associated with accidents. Each record represents a vehicle involved in an accident and includes attributes such as vehicle type, make, model, age of vehicle, engine capacity, and driver-related information. The combination of Accident_Index and Vehicle_Reference forms a composite key, establishing a relationship with the Accident table.

These tables are highly normalized and maintain referential integrity through primary and foreign key relationships. The structure is optimized for transactional operations but requires transformation for analytical purposes. This makes the SQL Server source ideal for demonstrating ETL processes such as joins, lookups, and data transformation.

3.3 CSV Source (Location Data)

To simulate integration from external systems, a CSV file named **Location_Dimension.csv** was introduced as a secondary data source. This file contains location-related attributes that enhance the geographical context of accident data.

The CSV file includes fields such as district, region, country, urban or rural classification, police force jurisdiction, and other location descriptors. These attributes are not fully normalized within the transactional database and are therefore maintained as a separate external source.

The use of a CSV file demonstrates the ability of the ETL process to handle semi-structured data formats. During the ETL process, this file is extracted using a Flat File Connection Manager in SSIS, and the data is integrated into the data warehouse through transformations and lookups. This approach reflects real-world scenarios where organizations often receive supplementary data from external systems in flat file formats, requiring proper parsing and integration into the central data warehouse.

3.4 Excel Source (Vehicle Category Data)

An additional data source in the form of an Excel file named **Vehicle_Category.xlsx** was incorporated to enrich vehicle-related information. This file provides classification details for vehicles, such as vehicle categories and grouping attributes that are not explicitly available in the main transactional dataset.

The Excel file serves as a reference or lookup dataset that enhances the dimensional model, particularly for the DimVehicle dimension. By incorporating this external source, the ETL process demonstrates the ability to integrate structured spreadsheet data into the warehouse. The Excel data is extracted using an Excel Connection Manager in SSIS and is combined with the vehicle data from the SQL Server source. This integration enables the creation of richer and more meaningful dimensions, supporting advanced analytical queries and reporting. The inclusion of an Excel source further strengthens the multi-source architecture of the solution and highlights the flexibility of SSIS in handling diverse data formats.

3.5 Data Source Integration Strategy

The integration of multiple data sources is achieved through a well-defined ETL strategy implemented using SQL Server Integration Services (SSIS). The primary objective of this strategy is to extract data from heterogeneous sources, transform it into a consistent format, and load it into the data warehouse while ensuring data quality and integrity.

The integration process follows these key steps:

- **Data Extraction**

Data is extracted from the SQL Server database, CSV file, and Excel file using appropriate connection managers. Each source is accessed independently, ensuring flexibility in handling different formats.

- **Data Transformation**

The extracted data is transformed using SSIS components such as Derived Columns,

Lookups, and Data Conversion transformations. These transformations standardize data formats, resolve inconsistencies, and enrich the data by combining attributes from multiple sources.

- **Data Integration**

Data from different sources is merged based on common keys such as Accident_Index and other business attributes. Lookup transformations are used to map source data to dimension tables, ensuring referential integrity in the data warehouse.

- **Data Loading**

The transformed and integrated data is loaded into the dimensional model, consisting of fact and dimension tables. The loading process follows a structured sequence, where dimension tables are populated before the fact table.

This integration strategy ensures that data from multiple heterogeneous sources is seamlessly combined into a unified data warehouse. It also demonstrates the capability of SSIS to handle complex ETL workflows, making the solution robust and scalable for analytical processing.

3.6 Data Source Summary

The following table summarizes the different data sources used in this project:

Source Name	Type	Description
Accident_Information	SQL Server	Accident-level transactional data
Vehicle_Information_Clean	SQL Server	Vehicle-level data with valid relationships
Location_Dimension.csv	CSV	Location hierarchy (District → Region → Country)
Vehicle_Category.xlsx	Excel	Vehicle classification and enrichment data

4. Solution Architecture

4.1 Architecture Overview

The solution architecture for this project is designed to support the complete lifecycle of data warehousing, from data extraction to analytical readiness. The architecture follows a layered approach, integrating multiple heterogeneous data sources into a centralized data warehouse using ETL processes implemented in SQL Server Integration Services (SSIS).

The architecture consists of three primary layers: the **data source layer**, the **ETL processing layer**, and the **data warehouse layer**. Each layer plays a distinct role in ensuring that raw operational data is transformed into structured, consistent, and analysis-ready information.

The **data source layer** includes a SQL Server database containing transactional accident and vehicle data, along with external files such as CSV and Excel that provide supplementary

information. These sources represent a realistic enterprise environment where data is distributed across multiple systems and formats.

The **ETL layer** is responsible for extracting data from these sources, performing necessary transformations, and loading it into the data warehouse. This layer handles tasks such as data cleansing, data type conversions, surrogate key generation, and integration of multiple data sources. The ETL processes are implemented using SSIS packages, ensuring scalability and maintainability.

The **data warehouse layer** stores the transformed data in a dimensional model, consisting of a central fact table (**FactVehicleAccident**) and several dimension tables (**DimDate**, **DimLocation**, **DimRoadEnvironment**, **DimVehicle**, **DimDriver**, and **DimSeverity**). This structure is optimized for analytical queries and reporting.

Additionally, the architecture incorporates an **accumulating fact update mechanism**, where transaction lifecycle attributes are updated using a separate ETL process. This enables tracking of process completion times and supports advanced analytical capabilities.

Overall, the architecture ensures efficient data integration, high data quality, and optimal performance for analytical workloads.

4.2 Architectural Diagram

The architectural diagram illustrates the flow of data from multiple source systems into the data warehouse through ETL processes. It visually represents how different components interact within the system.

The diagram should include the following layers:

1. Data Source Layer

- SQL Server Database (**RoadSafety_SourceDB**)
- CSV File (**Location_Dimension.csv**)
- Excel File (**Vehicle_Category.xlsx**)

2. ETL Layer (SSIS)

- SSIS Packages:
 - **01_Load_Dimensions.dtsx**
 - **02_Load_FactVehicleAccident.dtsx**
 - **03_Update_AccumulatingFact.dtsx**

3. Data Warehouse Layer

- SQL Server Database (**RoadSafety_DW_ETL**)
- Dimension Tables:
 - DimDate
 - DimLocation
 - DimRoadEnvironment
 - DimVehicle
 - DimDriver
 - DimSeverity

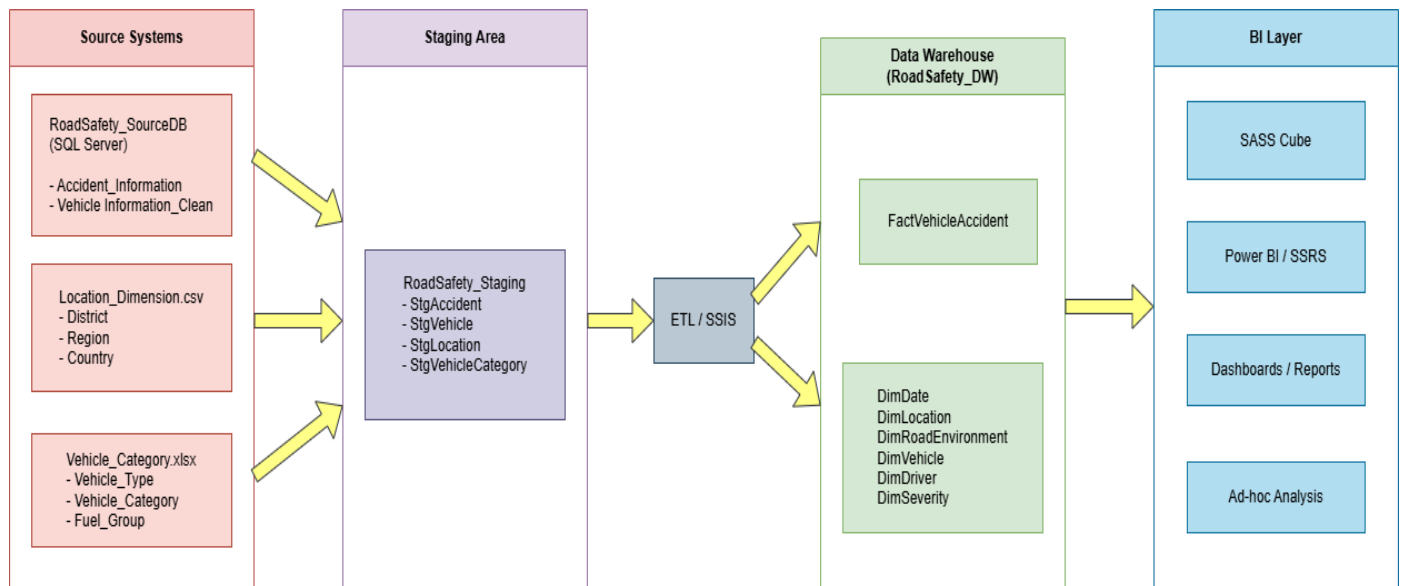
- Fact Table:
 - FactVehicleAccident

4. Staging Area

- Temporary table:
 - StgFactVehicleAccidentUpdate

5. Data Flow Direction

- Arrows showing:
 - Source → Staging → ETL → Data Warehouse → BI Layer



High-Level Data Warehouse and ETL Solution Architecture

4.3 Component Description

The solution architecture consists of several key components, each responsible for a specific function within the data warehousing process.

1. Source Systems

The source systems include a relational SQL Server database and external data files. The SQL Server database provides structured transactional data, while the CSV and Excel files provide supplementary and reference data. These sources collectively form the input layer of the system.

2. SSIS ETL Layer

The ETL layer is implemented using SQL Server Integration Services (SSIS) and consists of multiple packages designed to handle different stages of the data loading process.

- **01_Load_Dimensions.dtsx**
Responsible for loading and maintaining dimension tables. It performs data extraction from multiple sources and applies transformations such as lookups and Slowly Changing Dimension (Type 2) logic.
- **02_Load_FactVehicleAccident.dtsx**
Loads the fact table by integrating data from source tables and dimension tables. It uses lookup transformations to retrieve surrogate keys and ensures referential integrity.
- **03_Update_AccumulatingFact.dtsx**
Handles the update of accumulating fact table attributes, including completion timestamps and process duration calculations. This package reads from an external dataset and updates the fact table accordingly.

3. Staging Area

The staging area is implemented within the data warehouse database and is used for temporary storage of intermediate data. It plays a critical role in decoupling data extraction from final loading.

- **StgFactVehicleAccidentUpdate**
This staging table temporarily stores update records from the CSV file before they are applied to the fact table. It ensures controlled and consistent updates during the accumulating fact process.

4. Data Warehouse

The data warehouse is implemented in the **RoadSafety_DW_ETL** database and follows a dimensional modeling approach.

- **Fact Table**
 - FactVehicleAccident: Stores measurable and event-based data related to accidents
- **Dimension Tables**
 - DimDate: Time-related attributes
 - DimLocation: Geographical attributes
 - DimRoadEnvironment: Environmental conditions
 - DimVehicle: Vehicle details (with SCD Type 2 support)
 - DimDriver: Driver-related attributes
 - DimSeverity: Accident severity classification

This structure enables efficient querying and supports analytical processing.

5. Data Flow and Integration

Data flows from the source systems through the ETL layer into the data warehouse. During this process:

- Data is cleansed and standardized
- Keys are generated and mapped
- Multiple sources are integrated
- Fact and dimension tables are populated in a controlled sequence

This ensures high data quality and consistency across the warehouse.

5. Data Warehouse Design & Development

5.1 Dimensional Model Overview

The data warehouse for the Road Safety dataset is designed using a **dimensional modeling approach**, which restructures the normalized OLTP data into a format optimized for analytical processing and reporting. The dimensional model organizes data into a central fact table surrounded by multiple dimension tables, enabling efficient querying and aggregation.

The primary objective of this model is to support analytical queries related to accident trends, environmental conditions, vehicle characteristics, and driver attributes. By separating descriptive attributes into dimension tables and storing measurable data in a fact table, the model simplifies complex queries and improves performance.

The design follows best practices in data warehousing, including the use of surrogate keys, denormalized dimensions, and clearly defined relationships between fact and dimension tables. Additionally, historical tracking is incorporated through Slowly Changing Dimensions (Type 2), ensuring that changes in dimension data are preserved over time.

5.2 Star Schema Design

The data warehouse schema is implemented as a **star schema**, where a single central fact table is connected to multiple dimension tables. This design minimizes joins and enhances query performance, making it suitable for analytical workloads.

At the center of the schema is the **FactVehicleAccident** table, which stores transactional data related to road accidents. Surrounding this fact table are several dimension tables that provide descriptive context for analysis.

The star schema includes the following components:

- One central fact table:
 - **FactVehicleAccident**
- Multiple dimension tables:
 - DimDate
 - DimLocation
 - DimRoadEnvironment
 - DimVehicle
 - DimDriver
 - DimSeverity

Each dimension table is directly linked to the fact table using surrogate keys, forming a simple and efficient structure. This design enables users to perform multidimensional analysis such as analyzing accidents by time, location, vehicle type, or environmental conditions.

5.3 Fact Table – FactVehicleAccident

The **FactVehicleAccident** table is the core table of the data warehouse and stores measurable and event-based data related to road accidents.

Each record in the fact table represents a **vehicle involved in an accident**, making it a granular-level fact table that supports detailed analysis.

Key Characteristics

- **Grain of the fact table**
Each row represents one vehicle involved in a specific accident.
- **Primary Key**
 - FactKey (surrogate key)
- **Foreign Keys**
 - DateKey
 - LocationKey
 - RoadEnvKey
 - VehicleKey
 - DriverKey
 - SeverityKey
- **Degenerate Dimensions**
 - Accident_Index
 - Vehicle_Reference
- **Measures**
 - Number_of_Casualties
 - Number_of_Vehicles
 - Age_of_Vehicle
 - Engine_Capacity_CC
- **Accumulating Fact Attributes (Task 6)**
 - txn_id
 - accm_txn_create_time
 - accm_txn_complete_time
 - txn_process_time_hours

The fact table integrates data from multiple sources and dimensions, allowing for comprehensive analysis of accident events.

5.4 Dimension Tables

The dimension tables provide descriptive attributes that support filtering, grouping, and aggregation of fact data. Each dimension is designed to be denormalized for simplicity and performance.

5.4.1 DimDate

The **DimDate** table provides temporal context for accident data.

It includes attributes such as:

- FullDate
- DayNumber
- MonthNumber
- MonthName
- QuarterNumber

- YearNumber
- DayOfWeekName

This dimension enables time-based analysis, such as identifying accident trends by month, year, or day of the week.

5.4.2 DimLocation

The **DimLocation** table stores geographical information related to accident locations.

It includes attributes such as:

- Local_Authority_District
- Region
- Country
- Urban_or_Rural_Area
- Police_Force
- InScotland

This dimension allows analysis of accidents across different regions and administrative areas.

5.4.3 DimRoadEnvironment

The **DimRoadEnvironment** table captures environmental and road-related conditions at the time of the accident.

It includes attributes such as:

- Weather_Conditions
- Light_Conditions
- Road_Surface_Conditions
- Road_Type
- Junction_Detail
- Junction_Control
- Carriageway_Hazards
- Pedestrian_Crossing details
- Special_Conditions_at_Site
- Speed_limit

This dimension supports analysis of how environmental factors influence accident occurrence and severity.

5.4.4 DimVehicle (SCD Type 2)

The **DimVehicle** table stores vehicle-related attributes and is implemented as a **Slowly Changing Dimension (Type 2)**.

It includes attributes such as:

- Vehicle_Type
- make
- model
- Propulsion_Code
- Vehicle_Category
- Fuel_Group

- Was_Vehicle_Left_Hand_Drive

SCD Type 2 Implementation

- Tracks historical changes in vehicle attributes
- Uses:
 - EffectiveFromDate
 - EffectiveToDate
 - IsCurrent

When a change occurs in vehicle data, a new record is inserted while preserving the old record. This ensures that historical analysis reflects the correct context at the time of the event.

5.4.5 DimDriver

The **DimDriver** table stores driver-related attributes extracted from vehicle data.

It includes:

- Age_Band_of_Driver
- Sex_of_Driver
- Driver_Home_Area_Type
- Driver_IMD_Decile
- Journey_Purpose_of_Driver

This dimension enables analysis of driver behavior and demographics in relation to accidents.

5.4.6 DimSeverity

The **DimSeverity** table stores accident severity classifications.

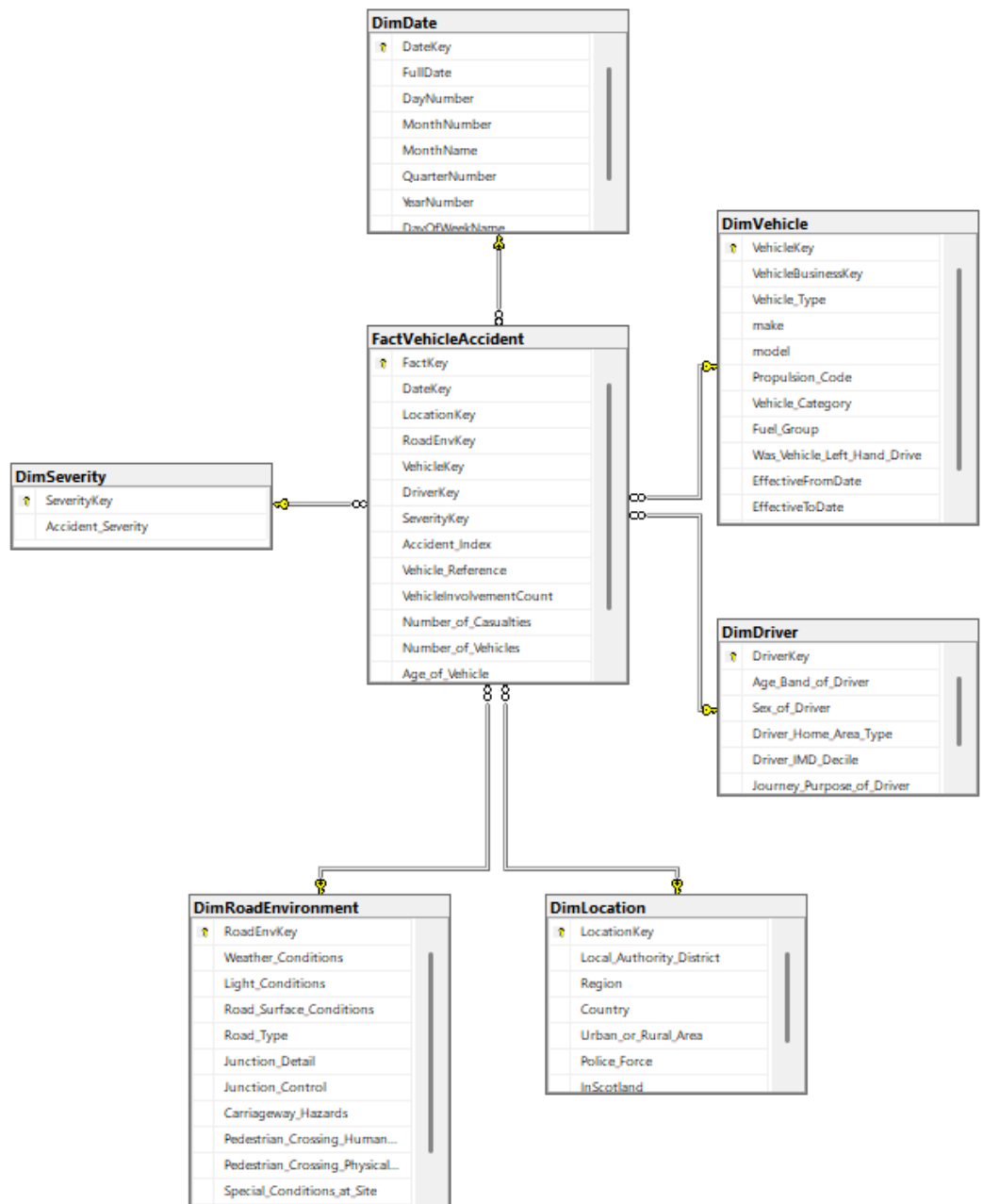
It includes:

- Accident_Severity

This dimension allows categorization and analysis of accidents based on severity levels such as slight, serious, or fatal.

5.5 Relational Diagram

The relational diagram represents the implementation of the star schema in the data warehouse.



Star Schema Relational Diagram of the Road Safety Data Warehouse

5.6 Design Assumptions

Several assumptions were made during the design and implementation of the data warehouse:

- Accident and vehicle data are assumed to be accurate and complete in the source system
- Each vehicle record is associated with exactly one accident
- Derived dimensions such as Location and RoadEnvironment are created by grouping related attributes from the source
- Vehicle attributes may change over time; therefore, SCD Type 2 is applied to preserve history
- Missing or inconsistent values are handled during the ETL process using default or standardized values
- Surrogate keys are used for all dimension tables to ensure consistency and improve performance
- The accumulating fact design assumes that completion times are updated after initial record creation

These assumptions ensure a consistent and practical design while supporting the analytical requirements of the data warehouse.

6. ETL Development

6.1 ETL Overview

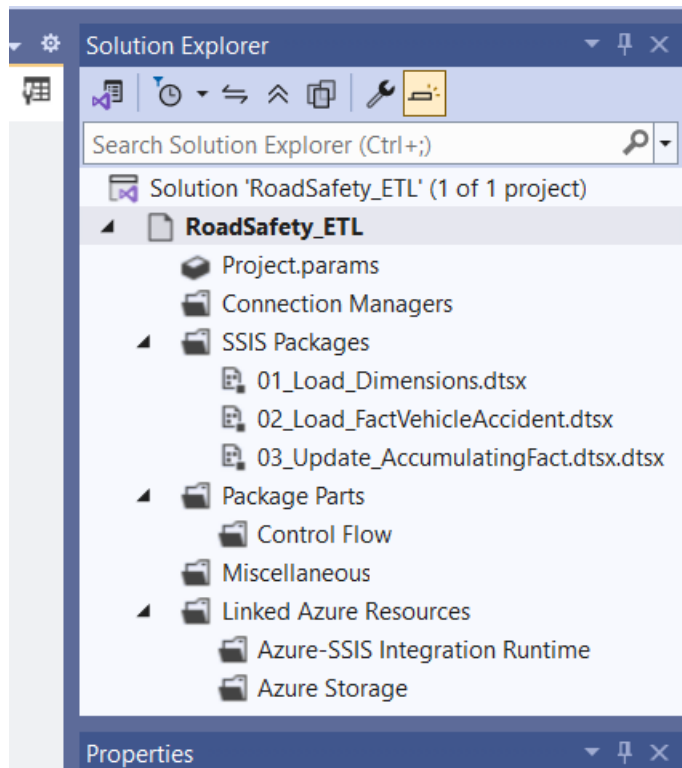
The Extract, Transform, and Load (ETL) process is a critical component of the data warehousing solution, responsible for integrating data from multiple heterogeneous sources into a unified dimensional model. In this project, the ETL processes are implemented using **SQL Server Integration Services (SSIS)**.

The ETL workflow is designed to extract data from a SQL Server database, CSV files, and Excel files, transform the data into a consistent and analysis-ready format, and load it into the data warehouse. The process ensures data quality, consistency, and referential integrity across all tables.

The ETL implementation is divided into two main packages:

- **01_Load_Dimensions.dtsx** – responsible for loading and maintaining dimension tables
- **02_Load_FactVehicleAccident.dtsx** – responsible for loading the fact table

The loading process follows a structured sequence where dimension tables are populated first, followed by the fact table. This ensures that all foreign key relationships are correctly established.



Overview of SSIS Packages for ETL Implementation

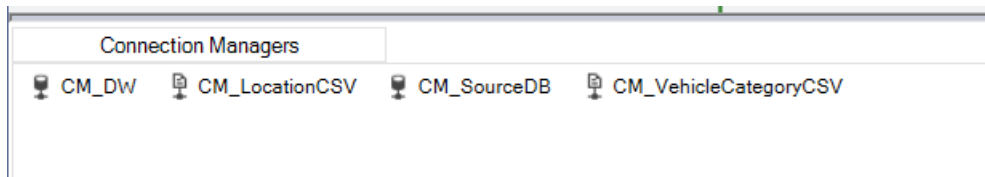
6.2 Connection Managers

Connection Managers are used in SSIS to establish connections to different data sources and destinations. In this project, multiple connection managers are configured to support data extraction and loading.

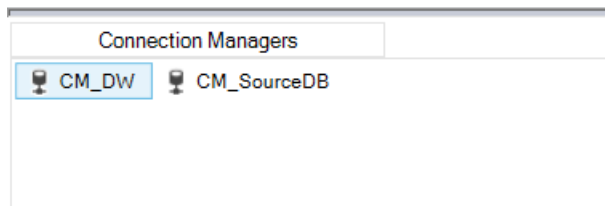
The following connection managers are used:

- **CM_SourceDB**
Connects to the SQL Server source database (**RoadSafety_SourceDB**) containing accident and vehicle data.
- **CM_DW**
Connects to the data warehouse database (**RoadSafety_DW_ETL**) where dimension and fact tables are stored.
- **FFS_LocationCSV**
Connects to the CSV file containing location data.
- **Excel_VehicleCategory**
Connects to the Excel file containing vehicle category information.

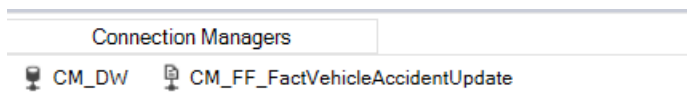
Each connection manager is configured with appropriate connection strings and authentication settings to ensure secure and efficient data access.



Connection Managers for 01_Load_Dimensions.dtsx



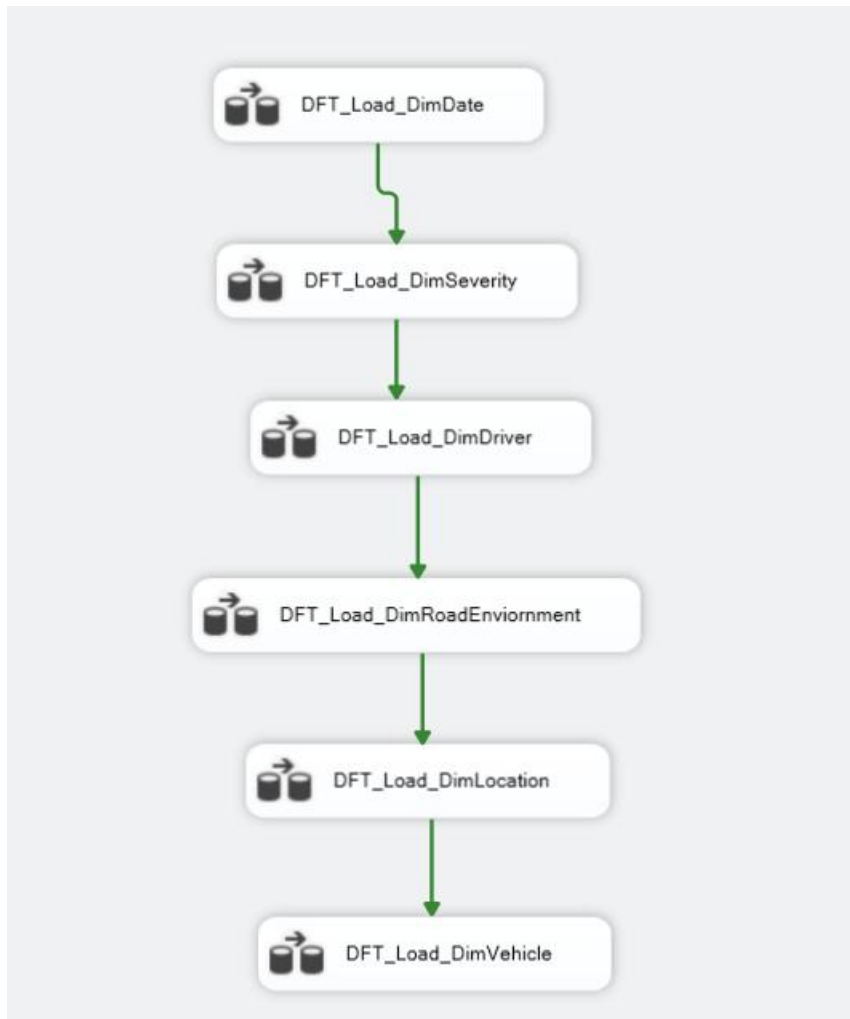
Connection Managers for Load FactVehicleAccident.dtsx



Connection Managers for 03_Update_AccumulatingFact.dtsx

6.3 Package 1: 01_Load_Dimensions.dtsx

This package is responsible for loading all dimension tables. It extracts data from multiple sources, performs transformations, and loads cleaned data into dimension tables.



Control Flow of Dimension Loading Package

6.3.1 Dimension Loading Strategy

The dimension loading process follows a structured approach to ensure consistency and data quality. Each dimension is loaded using a separate Data Flow Task, allowing modular and maintainable ETL design.

The process involves extracting data from source systems, performing necessary transformations such as data cleansing and standardization, and loading the results into dimension tables using surrogate keys. For the DimVehicle table, Slowly Changing Dimension (Type 2) logic is applied to track historical changes.

This modular strategy ensures that each dimension can be maintained independently while preserving data integrity across the warehouse.

6.3.2 Loading DimDate

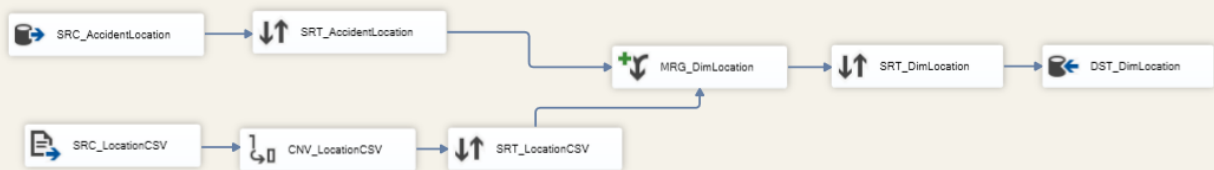
The DimDate table is populated with date-related attributes derived from accident dates. The ETL process extracts date values and generates additional attributes such as month, year, and day names.



Data Flow for Loading DimDate

6.3.3 Loading DimLocation

The DimLocation table is populated using data from the CSV source. The ETL process extracts location attributes and loads them into the dimension table.



Data Flow for Loading DimLocation from CSV Source

6.3.4 Loading DimRoadEnvironment

The DimRoadEnvironment table is populated using environmental attributes extracted from the accident data.

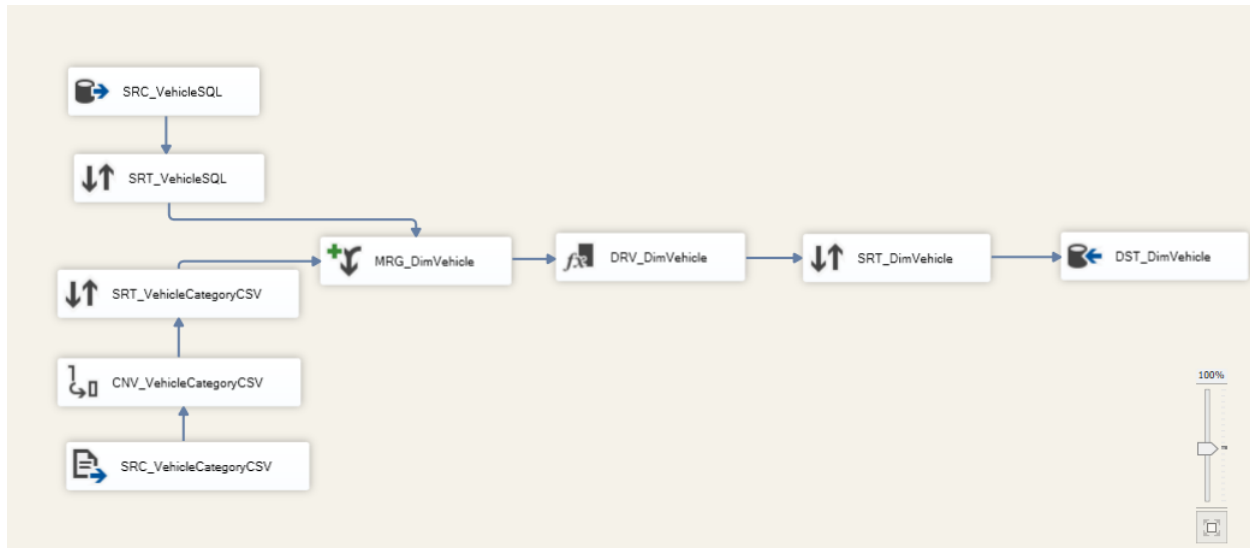


Data Flow for Loading DimRoadEnvironment

6.3.5 Loading DimVehicle (SCD Type 2)

The DimVehicle table is implemented as a Slowly Changing Dimension (Type 2) to track historical changes in vehicle attributes.

The ETL process detects changes in source data and inserts new records while preserving existing records with updated effective dates.



SCD Type 2 Implementation for DimVehicle

6.3.6 Loading DimDriver

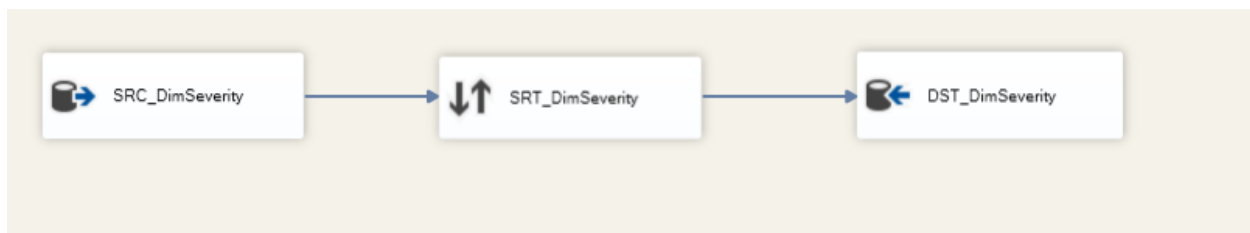
The DimDriver table is populated using driver-related attributes extracted from vehicle data.



Data Flow for Loading DimDriver

6.3.7 Loading DimSeverity

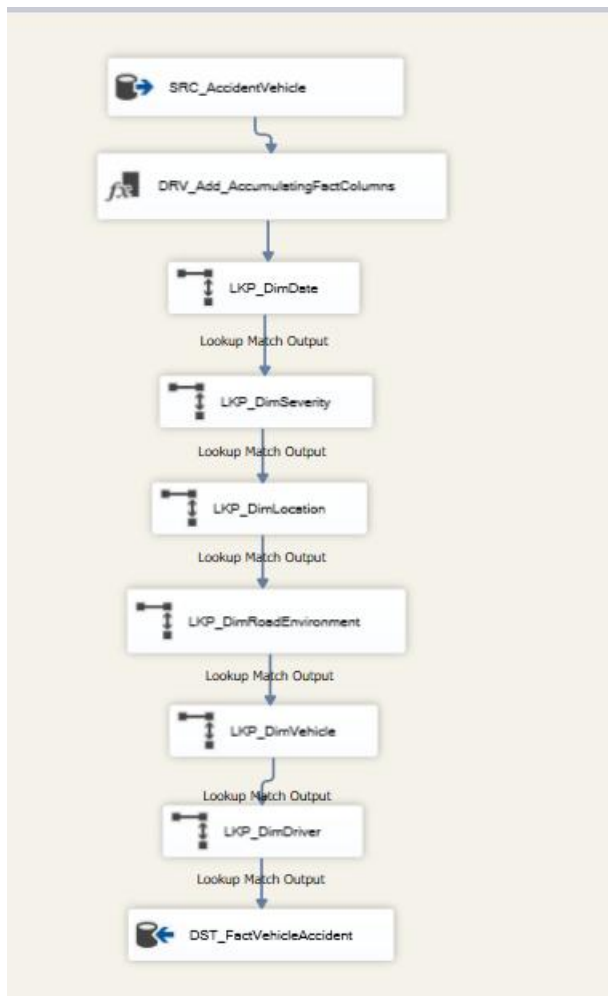
The DimSeverity table is populated using accident severity data.



Data Flow for Loading DimSeverity

6.4 Package 2: 02_Load_FactVehicleAccident.dtsx

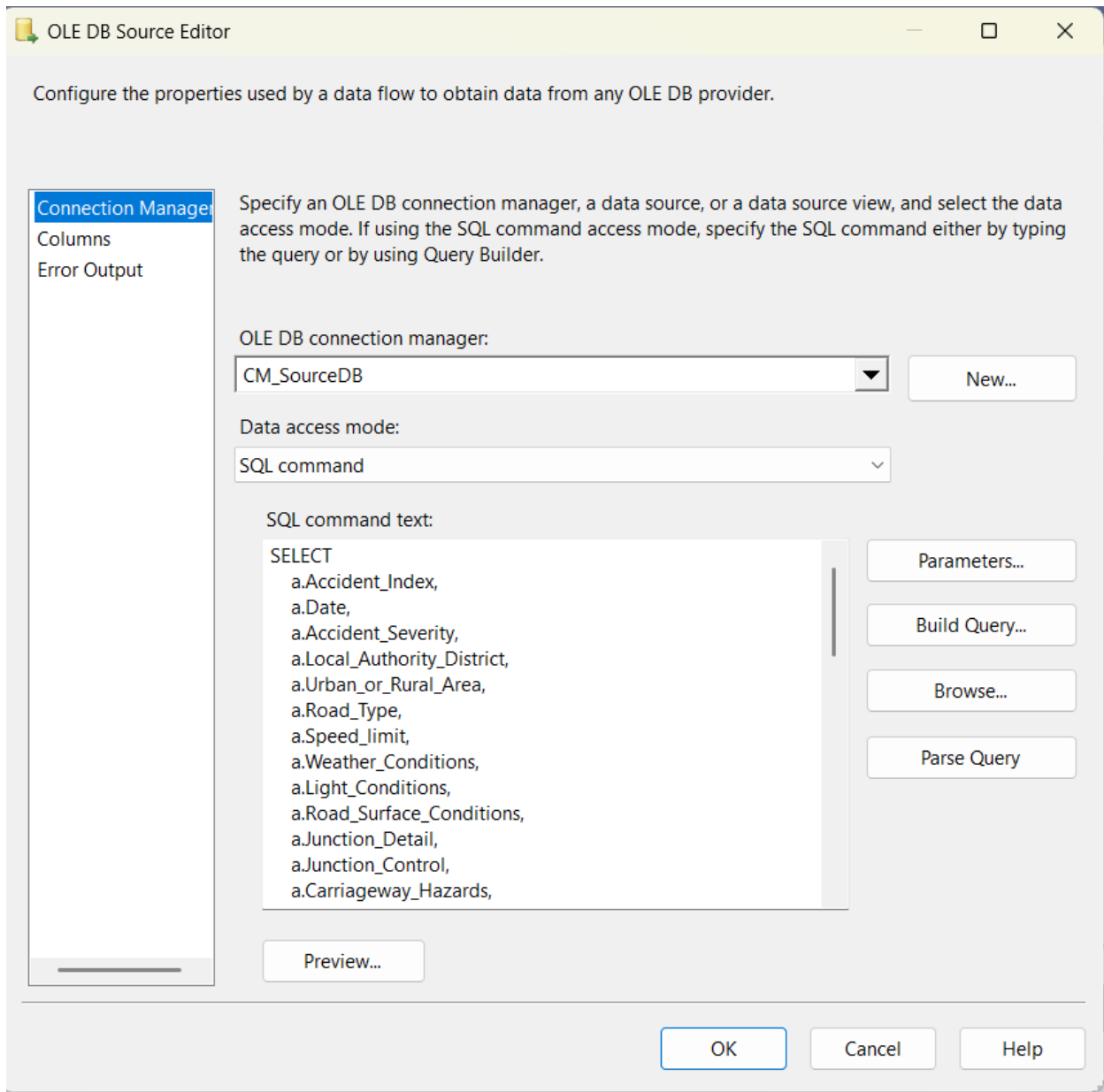
This package is responsible for loading the central fact table by integrating data from source systems and dimension tables.



Control Flow of Fact Loading Package

6.4.1 Source Data Extraction

The ETL process extracts data by joining accident and vehicle tables from the source database. This ensures that all relevant attributes are available for transformation and loading.



Source Data Extraction Query for Fact Table

6.4.2 Data Transformations

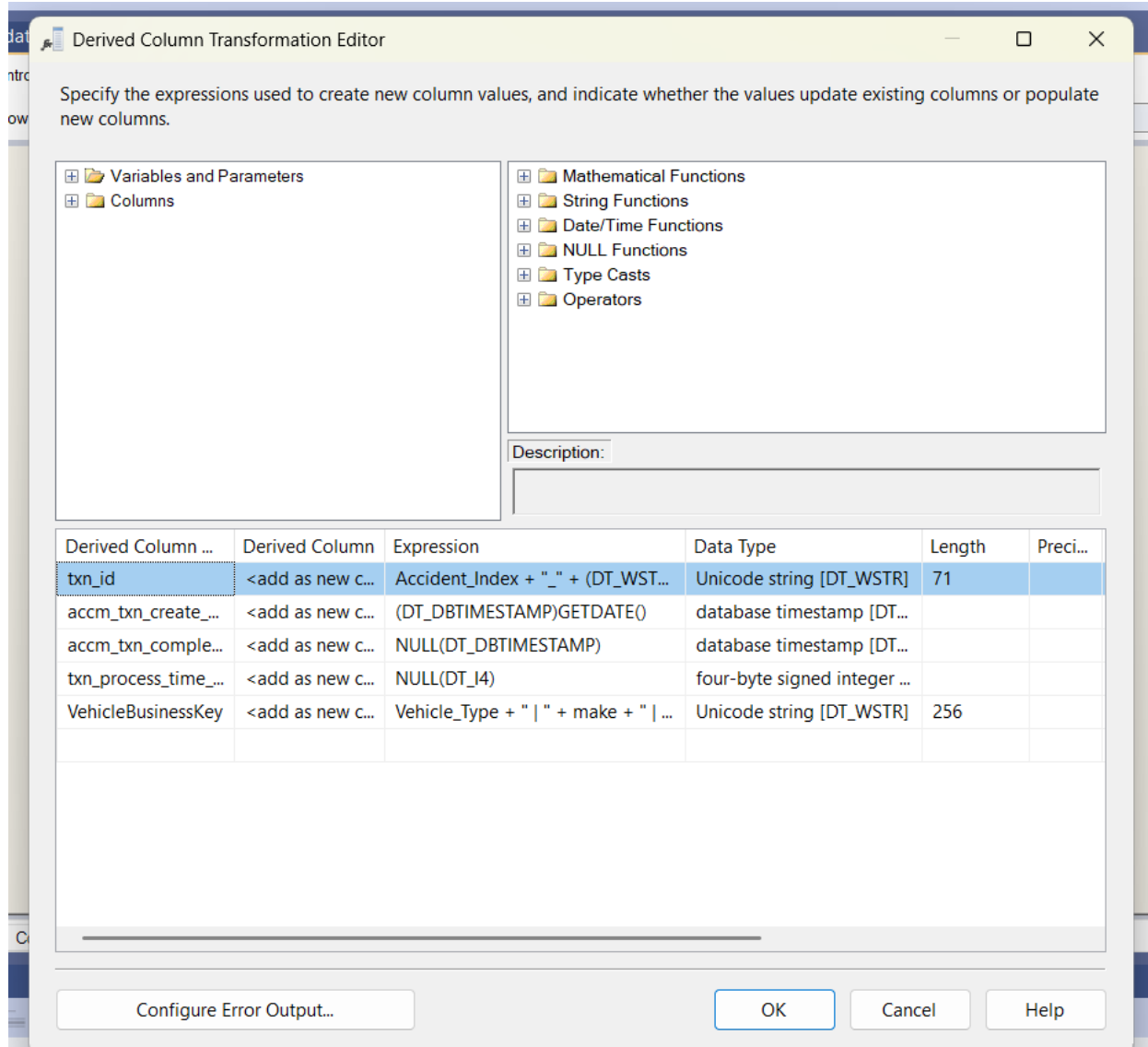
The transformation stage prepares data for loading into the fact table by applying necessary calculations and mappings.

Derived Columns

Derived columns are used to create new fields such as:

- txn_id
- accm_txn_create_time
- default values for accumulating fact attributes

These transformations ensure that required fields are available before loading.



Derived Column Transformation for Fact Table

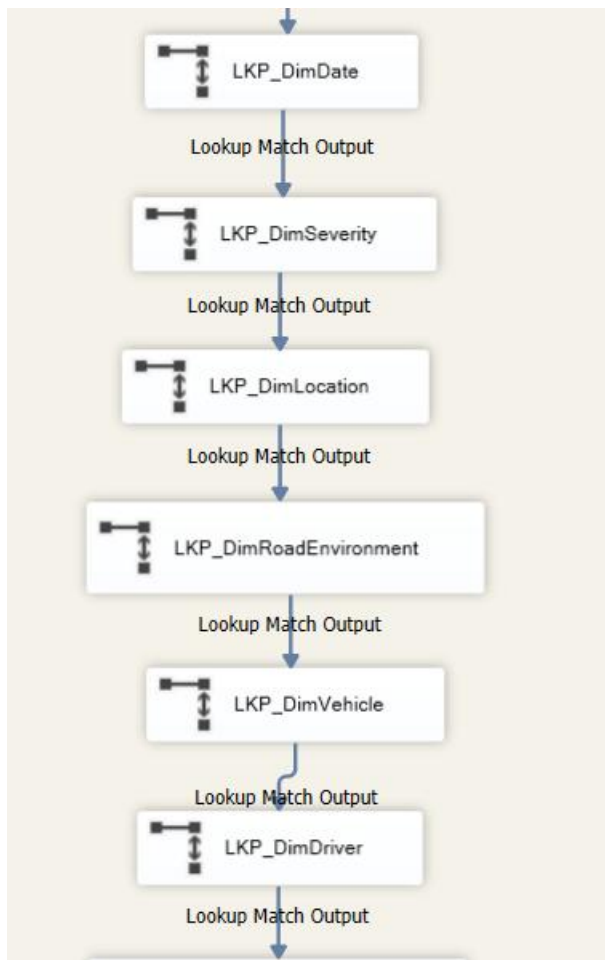
Lookups

Lookup transformations are used to retrieve surrogate keys from dimension tables.

The following lookups are implemented:

- DimDate
- DimLocation
- DimRoadEnvironment
- DimVehicle
- DimDriver
- DimSeverity

These ensure that the fact table maintains referential integrity.

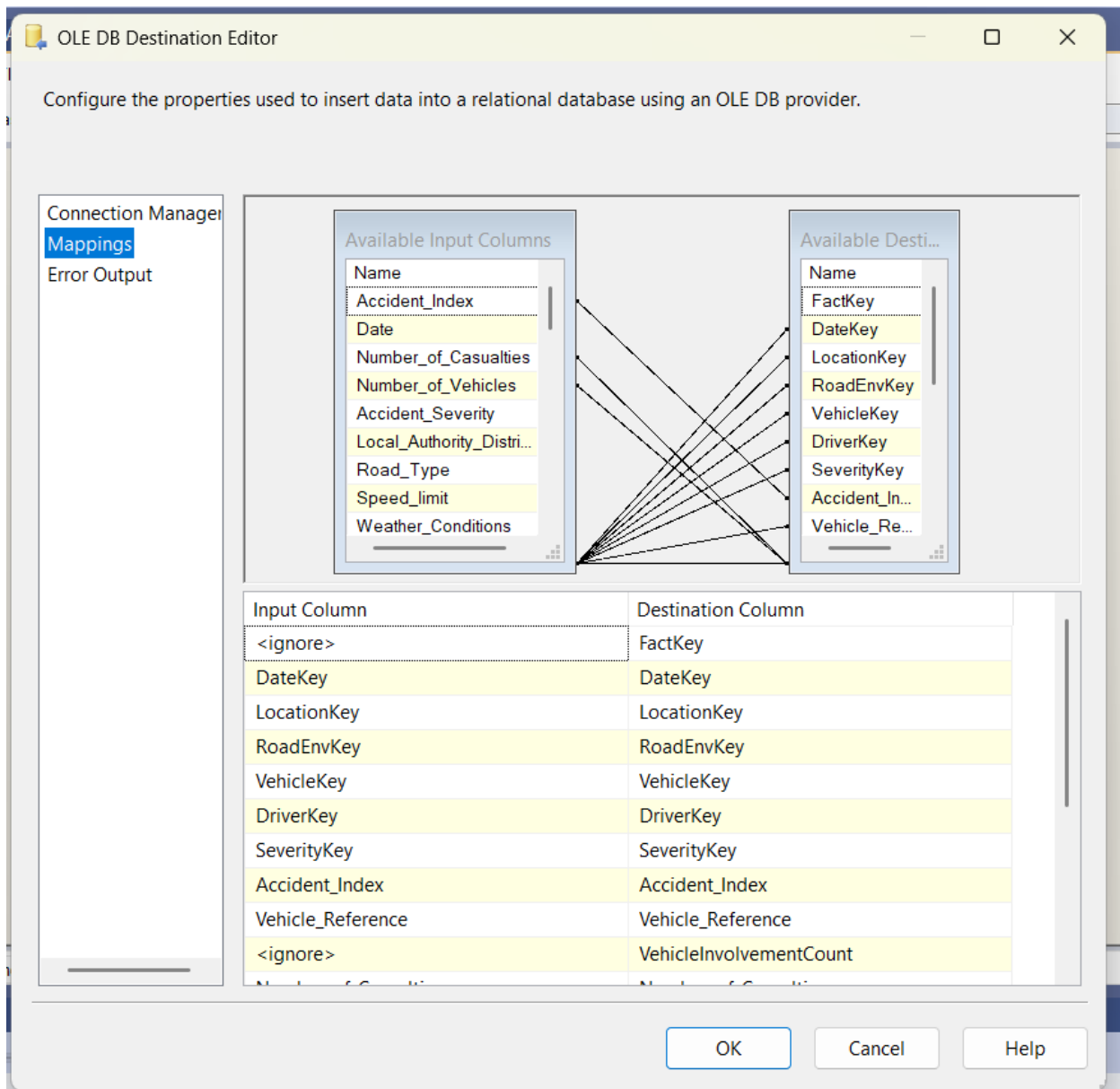


Lookup Transformations for Mapping Dimension Keys

6.4.3 Fact Table Loading Process

After transformations, the processed data is loaded into the FactVehicleAccident table using an OLE DB Destination.

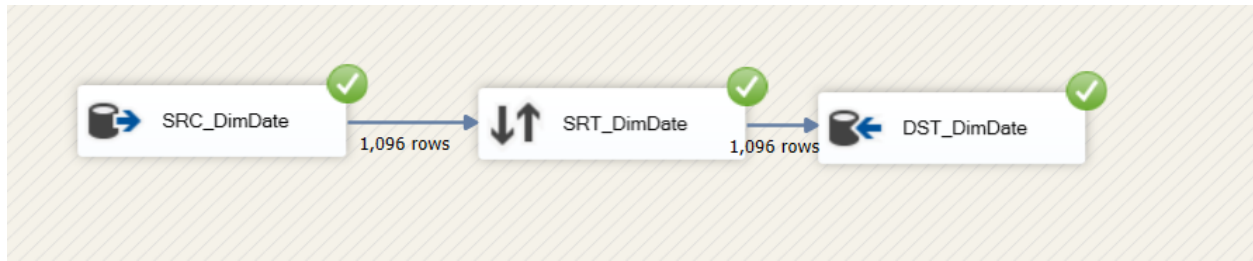
The loading process ensures that all foreign keys are correctly mapped and that measures and attributes are stored accurately.



Loading Data into FactVehicleAccident Table

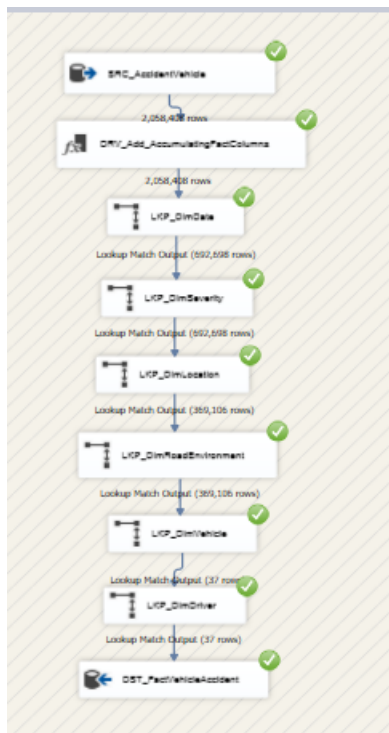
6.5 Execution Results

The ETL processes were successfully executed using three SSIS packages, ensuring the complete loading and updating of the data warehouse. The execution results confirm that all data flows were processed without errors and the expected number of records were loaded.



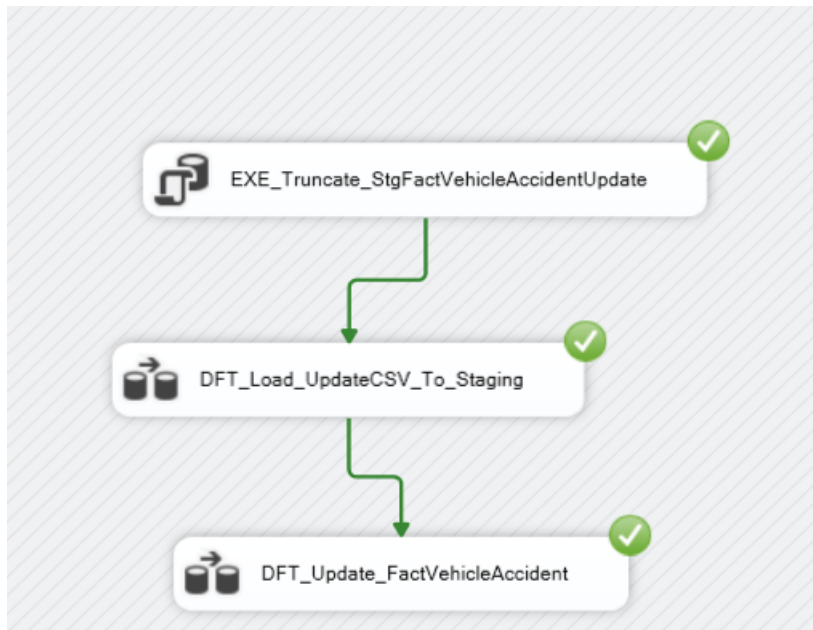
Successful Execution of Package 1 – Dimension Tables Loading

This figure shows the successful execution of the dimension loading package. All dimension tables were populated correctly, ensuring that surrogate keys and reference data were available for fact table loading.



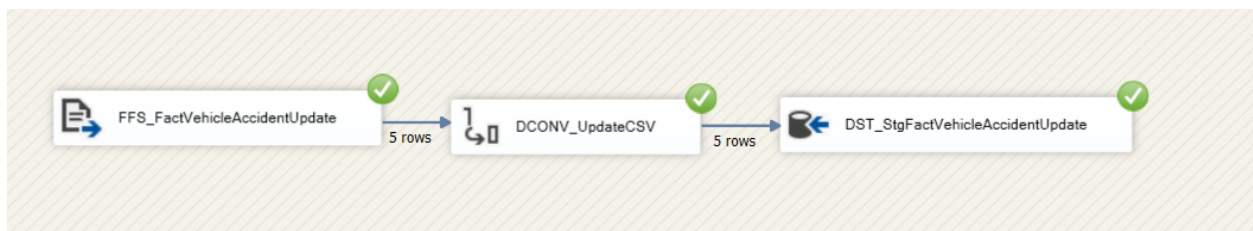
Successful Execution of Package 2 – Fact Table Loading with Row Counts

This figure illustrates the execution of the fact table loading process. A large number of records were successfully extracted, transformed, and loaded into the FactVehicleAccident table. Row counts displayed in the data flow confirm the volume of processed data.



Successful Execution of Package 3 – Accumulating Fact Table Update

This figure shows the successful execution of the accumulating fact update package. The process includes loading update data into a staging table and applying updates to the fact table using a stored procedure.



Total Number of Records Loaded into Fact Table

This figure presents the total number of records available in the FactVehicleAccident table after the ETL process, confirming successful data loading and integration.

7. Accumulating Fact Table Implementation

7.1 Concept of Accumulating Fact Table

An accumulating fact table is a specialized type of fact table used to track the lifecycle of a process or transaction over time. Unlike regular transactional fact tables, where records are

inserted once and remain unchanged, accumulating fact tables are updated as the state of a process evolves.

In this project, the accumulating fact concept is applied to track the lifecycle of accident-related records. Each transaction is initially created with a timestamp, and as the process progresses, additional attributes such as completion time and processing duration are updated. This allows the data warehouse to capture both the initial event and subsequent updates, enabling time-based analysis of process efficiency.

The implementation of an accumulating fact table enhances the analytical capabilities of the data warehouse by allowing users to measure the time taken between different stages of a transaction.

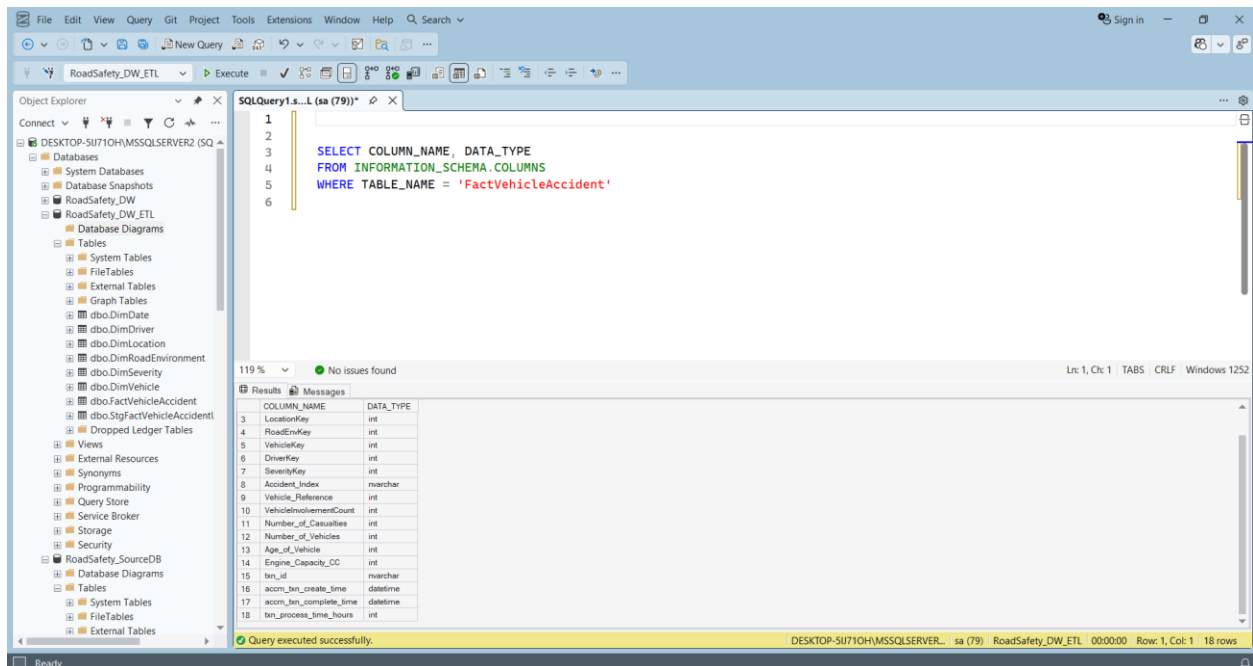
7.2 Fact Table Extension (New Columns)

To support the accumulating fact design, the existing fact table (**FactVehicleAccident**) was extended by adding additional attributes.

The following columns were introduced:

- **txn_id**
A natural key used to uniquely identify each transaction
- **accm_txn_create_time**
Stores the timestamp when the record is initially created
- **accm_txn_complete_time**
Stores the timestamp when the transaction is completed
- **txn_process_time_hours**
Stores the duration between creation and completion times

These columns enable tracking of transaction progress and calculation of processing time.

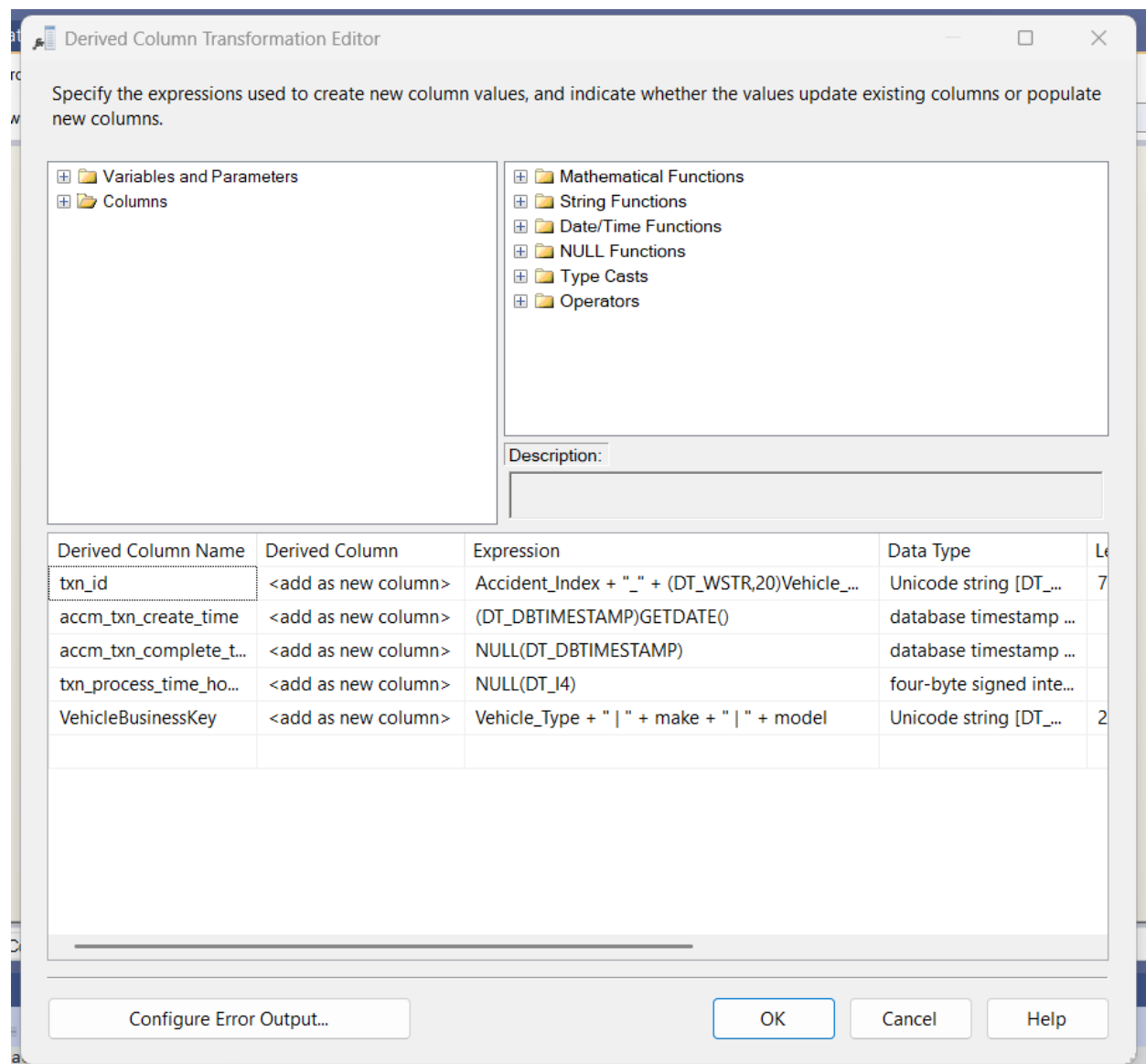


7.3 Initial Load Enhancements (txn_id & create time)

During the initial fact table loading process, additional transformations were introduced to populate the accumulating fact attributes.

- The **txn_id** was generated by concatenating Accident_Index and Vehicle_Reference, ensuring uniqueness at the transaction level
- The **accm_txn_create_time** was set using the system date (GETDATE()), capturing the record creation time
- The remaining attributes were initially set to NULL, to be updated later

This ensures that each record is initialized correctly before subsequent updates.



Derived Column Transformation for Accumulating Fact Attributes

7.4 Update Dataset Preparation (CSV File)

A separate dataset was prepared to simulate the completion of transactions. This dataset was created as a CSV file containing:

- txn_id
- accm_txn_complete_time

The dataset represents an external update source and includes timestamps that are later than the creation time.

This approach reflects real-world scenarios where updates to transactional data may come from external systems.

txn_id	accm_txn_complete_time
201401BS70841.1	3/24/2026 10:00
201401CW11755.1	3/25/2026 12:30
201401EK11007.2	3/26/2026 9:15
201401HT20394.2	3/27/2026 15:45
201401XD60617.1	3/28/2026 11:00

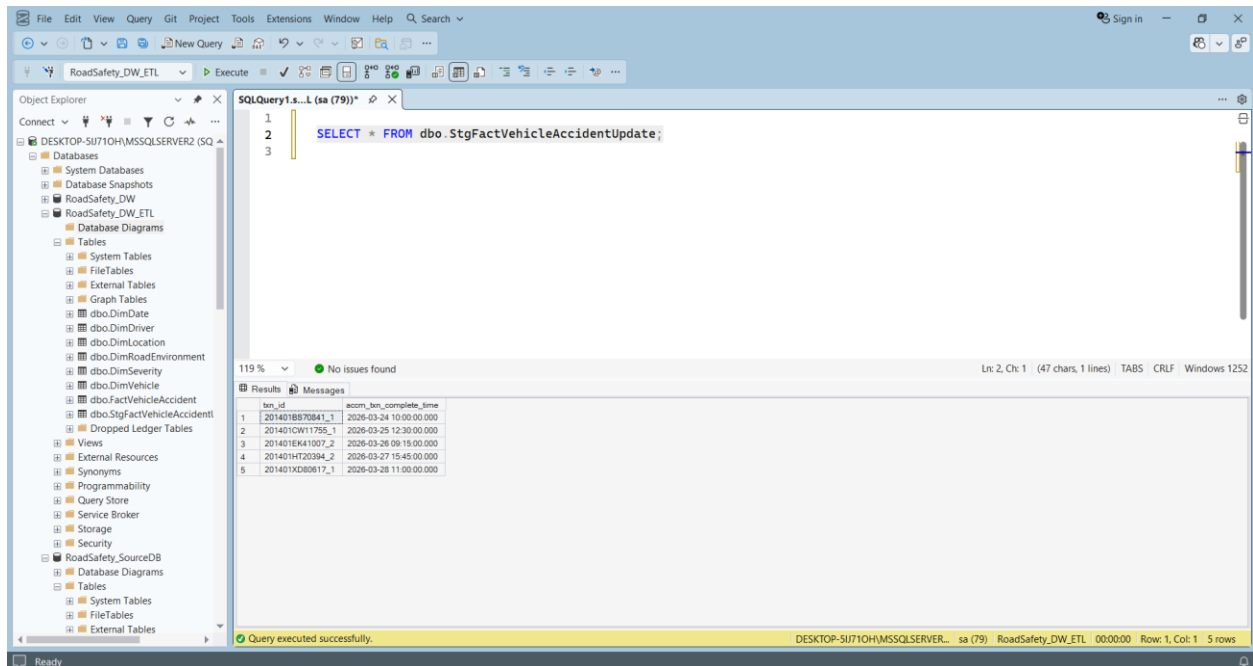
CSV Dataset for Updating Accumulating Fact Table

7.5 Staging Table Design

A staging table was created to temporarily store the update data before applying it to the fact table.

- Table Name: **StgFactVehicleAccidentUpdate**
- Columns:
 - txn_id
 - accm_txn_complete_time

The staging table acts as an intermediate layer, allowing controlled updates and improving data consistency during the ETL process.



Staging Table for Accumulating Fact Updates

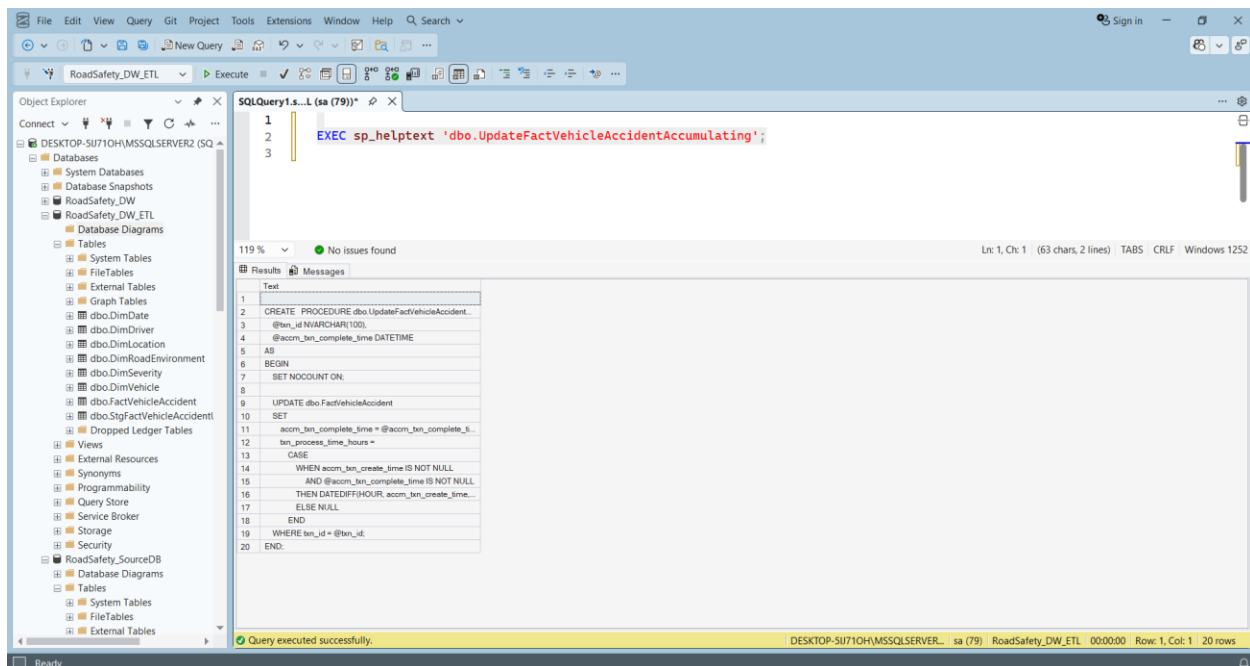
7.6 Stored Procedure Implementation

A stored procedure was developed to update the fact table based on the staging data.

The procedure performs the following:

- Updates **accm_txn_complete_time**
- Calculates **txn_process_time_hours** using DATEDIFF
- Ensures that only matching records are updated

This encapsulates the update logic and improves maintainability.



Stored Procedure for Updating Accumulating Fact Table

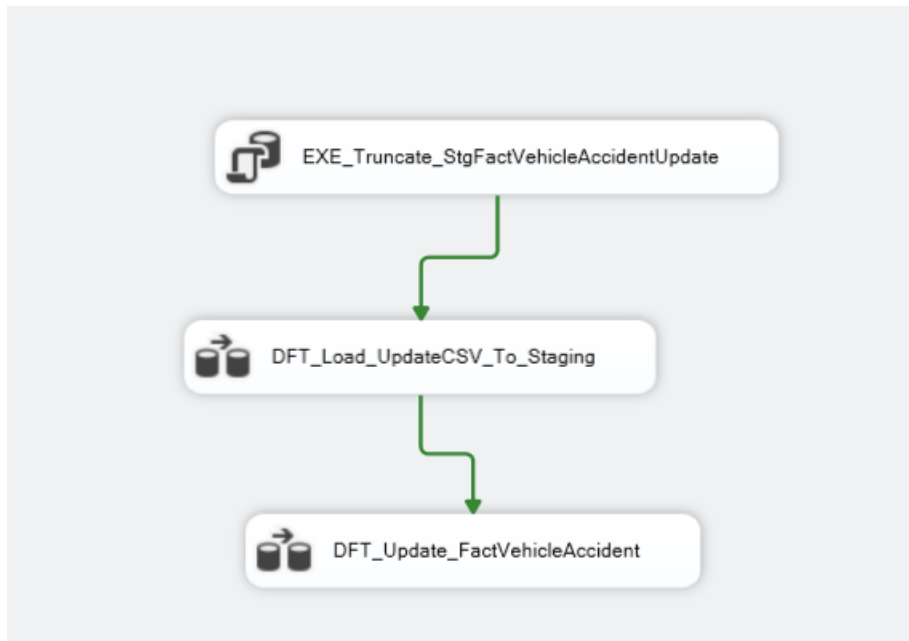
7.7 Package 3: 03_Update_AccumulatingFact.dtsx

This SSIS package is responsible for updating the accumulating fact table using external data.

7.7.1 Control Flow Design

The control flow consists of:

- Truncate staging table
- Load CSV data into staging
- Execute update logic



Control Flow of Accumulating Fact Update Package

7.7.2 Data Flow – Load CSV to Staging

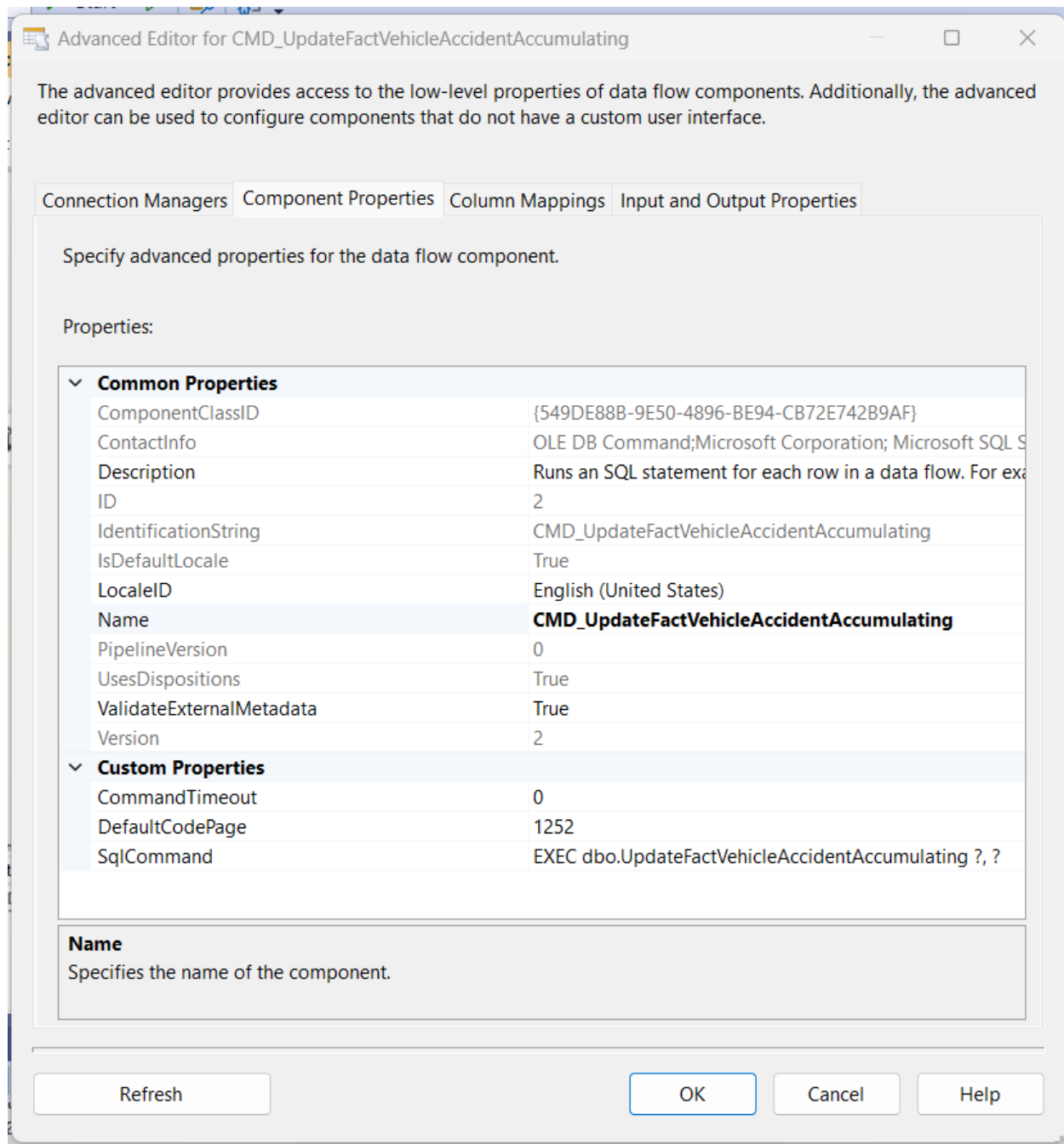
This data flow extracts data from the CSV file, performs data type conversions, and loads it into the staging table.



Data Flow for Loading CSV Data into Staging Table

7.7.3 Data Flow – Update Fact Table

This data flow reads from the staging table and uses an OLE DB Command to execute the stored procedure for updating the fact table.



OLE DB Command for Updating Fact Table

7.8 Validation and Results

7.8.1 Updated Fact Table Records

After executing the ETL package, the fact table was successfully updated with completion timestamps.

The screenshot displays the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the database structure for 'DESKTOP-SIJ710H\MSSQLSERVER2'. The central pane shows a SQL query being executed:

```

1
2
3 SELECT txn_id, accm_txn_complete_time, txn_process_time_hours
4 FROM dbo.FactVehicleAccident
5 WHERE accm_txn_complete_time IS NOT NULL;
6

```

The Results pane at the bottom shows the output of the query, which consists of 5 rows of data:

txn_id	accm_txn_complete_time	txn_process_time_hours
2014074049147_1	2026-03-26 10:00:00.000	45
201501EK41006_2	2026-03-27 10:00:00.000	69
201501EK040195_1	2026-03-28 10:00:00.000	93
201501EK040223_1	2026-03-29 10:00:00.000	117
201501HT20004_2	2026-03-30 10:00:00.000	141

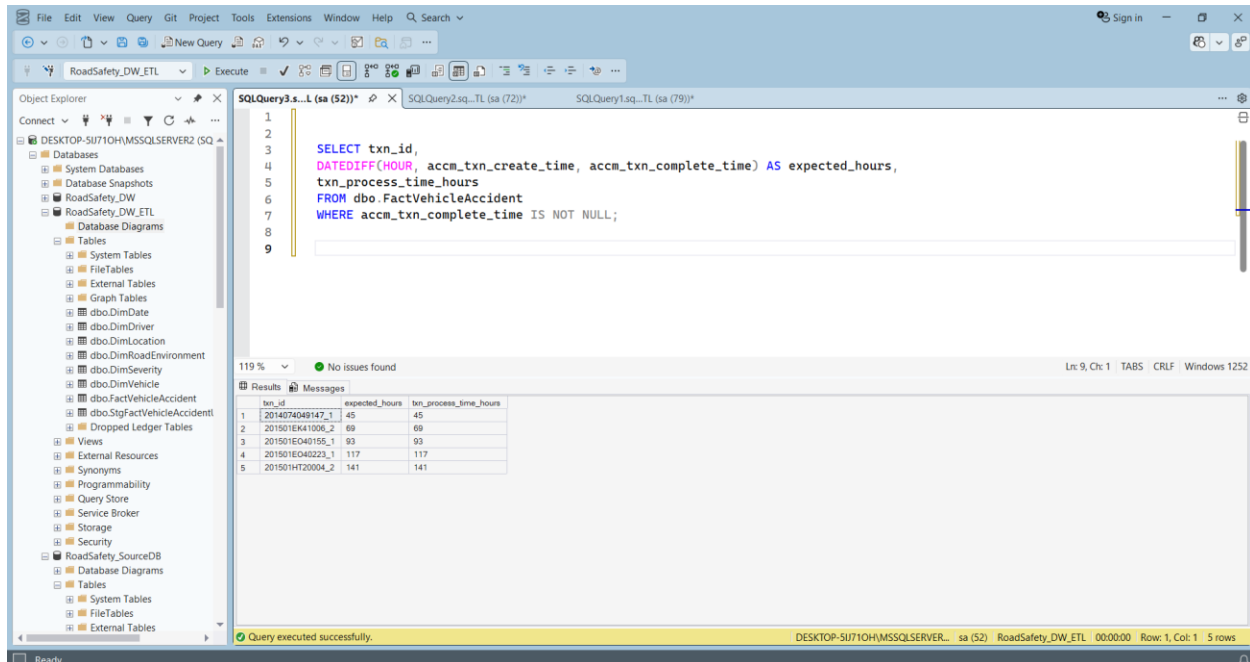
The status bar at the bottom indicates 'Query executed successfully.' and 'DESKTOP-SIJ710H\MSSQLSERVER... sa (52) RoadSafety_DW_ETL 00:00:00 Row: 1, Col: 1 5 rows'.

Updated Fact Table Records with Completion Time

This figure shows the updated fact table records after applying the accumulating fact update. The `accm_txn_complete_time` column has been successfully populated using the external update dataset.

7.8.2 Process Time Calculation Verification

The correctness of the process time calculation was verified by comparing stored values with calculated values using `DATEDIFF`.



Verification of Transaction Processing Time Calculation

This figure validates the correctness of the transaction processing time calculation. The expected hours computed using the DATEDIFF function match the stored txn_process_time_hours values, confirming accurate ETL logic.

8. Conclusion

This assignment successfully demonstrated the end-to-end implementation of a data warehousing solution using the UK Road Safety dataset. The project covered all key phases of the data warehousing lifecycle, including data source preparation, solution architecture design, dimensional modeling, and ETL development using SQL Server Integration Services (SSIS). A well-structured star schema was designed with a central fact table (*FactVehicleAccident*) and multiple dimension tables such as *DimDate*, *DimLocation*, *DimRoadEnvironment*, *DimVehicle*, *DimDriver*, and *DimSeverity*. The ETL process was implemented through multiple SSIS packages to extract, transform, and load data from heterogeneous sources into the data warehouse.

Advanced ETL concepts were applied, including the use of lookup transformations for surrogate key mapping, derived columns for data transformation, and Slowly Changing Dimension (Type 2) implementation for vehicle data. Furthermore, the accumulating fact table concept was successfully implemented by extending the fact table with transaction tracking attributes. A separate ETL pipeline was developed to update the accumulating fact table using an external dataset. The process ensured that transaction completion times were updated correctly, and processing durations were calculated accurately using time difference logic. Validation queries

confirmed that the computed processing time matched the expected values, demonstrating the correctness of the implementation.

Overall, this project provided practical experience in designing and implementing a scalable data warehouse solution. It highlights the importance of data integration, transformation, and validation in enabling reliable analytical reporting and decision-making.